TECNOLOGICO NACIONAL DE MEXICO

Technical Manual

About GameZoneRtx

By Alan Gómez Mireles

20130820
Mail: alanyahir63@gmail.com

Contact: 8714153150

Play games…

Advanced programming topics

May 17 , 2022

INDEX

Contenido

Introduction 2
Objective 2
System Requierements 2
Hardware 2
Software 2
Other requierements 2
Processing 2
How to play the Snake Game? 5
How to play 3 in a Row? 5
Necessary files: 5
¿How to create a program that this type? 6
Código index.hmtl 8
Código del index.css: 9
Snake Game code: 16
3 in a Row code: 21
Glossary of Concepts 30
Suggestions 31
Tags html: 31
References 31

Introduction

Como parte del proyecto unidad 3 de la materia Tópicos avanzados de programación, se nos encargó la elaboración de un programa cliente-servidor, a lo cual surge como respuesta mi proyecto ZoneGameRtx.

Objective

El objetivo de este manual es informar al usuario interesado en la realización de un programa usando servlet y servidores locales o remotos, los conceptos, pasos, clases y cosas necesarias para su realización, además de una guía de pasos para saber cómo realizar este programa en específico.

System Requirements

Los requerimientos necesarios del sistema son:

Hardware

- Teclado, mouse, monitor
- Procesador 1.4 Ghz mínimo
- Memoria RAM min 2GB

Software

- 15 mb de espacio en memoria
- Java 8.0

Other requirements

- Tener instalado Apache Tomcat 8.0.27.0
- Tener instalado Netbeans 8.2
- Tener instalado un navegador
- Opcional (Xampp control panel v3.3.0)

Processing

Para entrar al programa es necesario descargar el archivo del programa, el cual contiene un archivo de tipo .war que contendrá nuestro index al igual que nuestros juegos, hechos para html.

Todo se contiene en la carpeta dist al igual que en la dirección principal.

Una vez ahí, abriremos nuestro archivo index.html que nos dirigirá a nuestra página principal.

Nuestra página principal se verá de la siguiente forma:

Una vez ahí, elegiremos nuestro juego preferido, pulsando sobre los círculos debajo de los títulos, y después pulsaremos el botón jugar.

El cuál nos dirigirá a nuestro juego escogido:

Juego de la serpiente (Snake Game)

Juego del gato (3 in a Row)

Los html de los juegos, se deben de importar de la carpeta de descarga, o en caso de querer realizarlos, importar sus html.
El archivo de index.css se debe importar igual de la carpeta de descarga, o en caso de realizar sus propios juegos, evaluar su diseño e importar sus debidos archivos.
Se debe crear el paquete Servlet, y crear una clase de tipo servlet, la cual nos contendrá la plantilla para realizar un httppServlet.
A continuación los códigos:

Código index.hmtl

```html
<!DOCTYPE html>
<!-- Predefinir el idioma -->
<html lang="en">
    <head>
        <!-- En head se ubican las propiedades principales, tamaño predefinido --
        <meta charset="UTF-8">
        <meta http-equiv="X-UA-Compatible" content="IE=edge">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <!-- Titulo de la pagina -->
        <title>Game Zone Rtx</title>
        <!-- Asociar css (hoja de estilo) a pagina principal -->
        <link rel="stylesheet" href="index.css">
    </head>
    <body>

        <!-- Clases de Css -->
        <div class="titulo">
            <h1>--GameZoneRtx--
                <p>Escoja su juego preferido:
            </h1>
            <br>
        </div>
```

```html
        <br>
        <!-- Relacionamos el servlet para enviar los datos con el method get -->
        <form action="Servlet" method="get">

            <div class="bloque juegos">

                <!-- Links para tomar las imagenes -->
                <img src="https://icons.iconarchive.com/icons/google/noto-emoji-animals-nature/128/22285-snake-icon.png" al
                <h2>SnakeGame</h2>
                <!-- Tomar valor del radio para mandar a servlet -->
                <input type="radio" name ="juego" value="SnakeGame">
            </div>

            <div class="bloque juegos">

                <!-- Links para tomar las imagenes -->
                <img src="https://cdn-icons-png.flaticon.com/512/566/566294.png" alt="">
                <h2>3 in a Row</h2>
                <!-- Tomar valor del radio para mandar a servlet -->
                <input type="radio" name ="juego" value="3inaRow">
            </div>
```

```html
46            <!-- Boton para enviar los datos al servlet -->
47            <input type="submit"  class="bloque boton" value="Jugar">
48
49            <!-- Burbujas para dar diseño a la página -->
              <div class="burbujas">
51                <div class="burbuja"></div>
52                <div class="burbuja"></div>
53                <div class="burbuja"></div>
54                <div class="burbuja"></div>
55                <div class="burbuja"></div>
56                <div class="burbuja"></div>
57                <div class="burbuja"></div>
58                <div class="burbuja"></div>
59                <div class="burbuja"></div>
60                <div class="burbuja"></div>
61            </div>
62
        </body>
64  </html>
```

Código del index.css:

```css
1   /* Dar fondo */
2   body{
3       background-color: black;
4   }
5
6   /* Clase titulo */
7   .titulo
8   {
        background-image: radial-gradient(circle at 50% -20.71%, #d1fff8 0, #befff6 10%
10          , #a7fff4 20%, #8dfff3 30%, #6cfbf2 40%, #3cf2f2 50%, #00e8f3 60%, #00dff5 7
11          #00d6f9 80%, #00cefd 90%, #00c6ff 100%);
12      text-align: center;
13      margin:  20px 500px;
14      padding: 10px;
15      border-radius: 120px;
16      /* Movimiento */
17      animation:movimiento 4s linear infinite;
18  }
```

```
20  /* a */
21  a
22  {
23      text-decoration: none;
24  }
25
26  /* Juegos */
27  .juegos{
28      animation: movimiento 2.5s linear infinite;
29  }
30
31  /* Bloques */
32  .bloque
33  {
34
35      background-image: radial-gradient(circle at 50% -20.71%, #fffe80 0,
36          #ffff7b 6.25%, #ffff78 12.5%, #f0ff77 18.75%, #daff77 25%, #c1ff79 31.25%,
37          #a4ff7c 37.5%, #83fd81 43.75%, #57f988 50%, #00f591 56.25%, #00f19c 62.5%, #00eda9 68.7
38          #00e9b8 75%, #00e6c8 81.25%, #00e2d9 87.5%, #00dfeb 93.75%, #00dcfd 100%);
39
40      text-align: center;
41      display: inline-block;
42      margin:  20px 200px 20px 270px;
43      padding: 20px 30px;
44      border-radius: 100px;
45
46  }
47
48  /* Bloque hover : pasar mouse para arriba y cambia de color */
49  .bloque:hover{
50      background-image: linear-gradient(180deg, #ff597b 0, #ff4d85 8.33%,
51          #ff3f8f 16.67%, #ff3299 25%, #f323a4 33.33%, #e415ae 41.67%, #d109b8 50%,
52          #ba0cc2 58.33%, #9f1bcd 66.67%, #7d28d7 75%, #4c34e0 83.33%, #003ee9 91.67%, #0046ef 100
53      transform: scale(1.2);
54      transition-property: all;
55      transition-duration: 0.3s;
56      transition-timing-function: linear;
57
58  }
```

```
60    /* Tamaño de imagen */
61    img{
62        max-width: 100px;
63        max-height: 100px;
64        visibility: visible;
65    }
66
67
68    /* Letra de boton */
69    .boton
70    {
71        font-size: 20px;
72        margin: 20px 630px;
73    }
74
75    /* Eliminar bordes */
76    *{
77        margin: 0;
78        padding: 0;
79        box-sizing: border-box;
```

```
85    .burbuja{
86        border-radius: 50%;
87        background: rgb(0, 207, 243);
88        opacity: .3;
89
90        position: absolute;
          bottom: -150;
92
93        animation: burbujas 3s linear infinite ;
94    }
95
96    .burbuja:nth-child(1){
97        width: 80px;
98        height: 80px;
99        left: 5%;
100       animation-duration: 3s;
101       animation-delay: 3s;
102   }
```

```
104 .burbuja:nth-child(2){
105     width: 100px;
106     height: 100px;
107     left: 35%;
108     animation-duration: 3s;
109     animation-delay: 5s;
110 }
111
112 .burbuja:nth-child(3){
113     width: 20px;
114     height: 20px;
115     left: 15%;
116     animation-duration: 1.5s;
117     animation-delay: 7s;
118 }
119
120 .burbuja:nth-child(4){
121     width: 50px;
122     height: 50px;
123     left: 90%;
124     animation-duration: 6s;
```

```
.burbuja:nth-child(5){
    width: 70px;
    height: 70px;
    left: 65%;
    animation-duration: 3s;
    animation-delay: 1s;
}

.burbuja:nth-child(6){
    width: 20px;
    height: 20px;
    left: 50%;
    animation-duration: 4s;
    animation-delay: 5s;
}

.burbuja:nth-child(7){
    width: 20px;
    height: 20px;
    left: 50%;
    animation-duration: 4s;
    animation-delay: 5s;
}

.burbuja:nth-child(8){
    width: 100;
    height: 100px;
    left: 52%;
    animation-duration: 5s;
    animation-delay: 5s;
```

```
160 .burbuja:nth-child(9){
161     width: 65px;
162     height: 65px;
163     left: 51%;
164     animation-duration: 3s;
165     animation-delay: 2s;
166 }
167
168 .burbuja:nth-child(10){
169     width: 40px;
170     height: 40px;
171     left: 35%;
172     animation-duration: 3s;
173     animation-delay: 4s;
174 }
175
176 /* Animaciones de las burbujas */
177 @keyframes burbujas{
178     0%{
179         bottom: 0;
180         opacity: 0;
181     }
182     30%{
183         transform: translateX(30px);
184     }
185     50%{
186         opacity: .4;
187     }
```

```
176    /* Animaciones de las burbujas */
177    @keyframes burbujas{
178        0%{
179            bottom: 0;
180            opacity: 0;
181        }
182        30%{
183            transform: translateX(30px);
184        }
185        50%{
186            opacity: .4;
187        }
188        100%{
189            bottom: 100vh;
190            opacity: 0;
191        }
192    }
193
194    /* Movimiento de las burbujas */
195    @keyframes movimiento{
196        0%{
197            transform: translateY(0);
198        }
199        50%{
200            transform: translateY(30px);
201        }
202        100%{
203            transform: translateY(0);
204        }
205    }
```

Snake Game code:

Código

```
<!DOCTYPE html>
<!--
Juego de la serpiente en Html
-->
<html>
  <head>
    <title>Snake Game</title>
    <style>
      body {
        background: #444;
        text-align: center;
        margin: 64px 0 0 0;
      }
      canvas {
        border: solid 8px white;
        background: linear-gradient(
          0deg,
          rgba(0, 102, 204, 1.0) 0%,
          rgba(125, 60, 152, 1.0) 100%
          );
      }
    </style>
  </head>
  <body>
    <h1>"Snake Game"</h1>
    <!-- Los canvas se utilizan para dibujar gráficos a través de sencuencias de comandos -->
    <canvas width="500" height="500"></canvas>

    <!-- Los scripts nos ayudan a hacer referencia a un ejecutable en un documento html -->
    <script>
      const STATE_RUNNING = 1;
      const STATE_LOSING = 2;

      const TICK = 80;
      const SQUARE_SIZE = 10;
      const BOARD_WIDTH = 50;
      const BOARD_HEIGHT = 50;
      const GROW_SCALE = 10;
      const DIRECTIONS_MAP = {
        'A': [-1, 0],
```

```
    'D': [1, 0],
    'S': [0, 1],
    'W': [0, -1],
    'a': [-1, 0],
    'd': [1, 0],
    's': [0, 1],
    'w': [0, -1],
};

let state = {
    canvas: null,
    context: null,
    snake: [{x: 0, y: 0}],
    direction: {x: 1, y: 0},
    prey: {x: 0, y: 0},
    growing: 0,
    runState: STATE_RUNNING
};

function randomXY() {
    return {
        x: parseInt(Math.random() * BOARD_WIDTH),
        y: parseInt(Math.random() * BOARD_HEIGHT)
    };
}

function tick() {
    const head = state.snake[0];
    const dx = state.direction.x;
    const dy = state.direction.y;
    const highestIndex = state.snake.length - 1;
    let tail = {};
    let interval = TICK;

    Object.assign(tail,
            state.snake[state.snake.length - 1]);

    let didScore = (
            head.x === state.prey.x
            && head.y === state.prey.y
            );

    if (state.runState === STATE_RUNNING) {
```

```
        for (let idx = highestIndex; idx > -1; idx--) {
            const sq = state.snake[idx];

            if (idx === 0) {
                sq.x += dx;
                sq.y += dy;
            } else {
                sq.x = state.snake[idx - 1].x;
                sq.y = state.snake[idx - 1].y;
            }
        }
    } else if (state.runState === STATE_LOSING) {
        interval = 10;

        if (state.snake.length > 0) {
            state.snake.splice(0, 1);
        }

        if (state.snake.length === 0) {
            state.runState = STATE_RUNNING;
            state.snake.push(randomXY());
            state.prey = randomXY();
        }
    }

    if (detectCollision()) {
        state.runState = STATE_LOSING;
        state.growing = 0;
    }

    if (didScore) {
        state.growing += GROW_SCALE;
        state.prey = randomXY();
    }

    if (state.growing > 0) {
        state.snake.push(tail);
        state.growing -= 1;
    }

    requestAnimationFrame(draw);
    setTimeout(tick, interval);
}
```

```
function detectCollision() {
   const head = state.snake[0];

   if (head.x < 0
         || head.x >= BOARD_WIDTH
         || head.y >= BOARD_HEIGHT
         || head.y < 0
         ) {
      return true;
   }

   for (var idx = 1; idx < state.snake.length; idx++) {
      const sq = state.snake[idx];

      if (sq.x === head.x && sq.y === head.y) {
         return true;
      }
   }

   return false;
}

function drawPixel(color, x, y) {
   state.context.fillStyle = color;
   state.context.fillRect(
         x * SQUARE_SIZE,
         y * SQUARE_SIZE,
         SQUARE_SIZE,
         SQUARE_SIZE
         );
}

function draw() {
   state.context.clearRect(0, 0, 500, 500);

   for (var idx = 0; idx < state.snake.length; idx++) {
      const {x, y} = state.snake[idx];
      drawPixel('#22dd22', x, y);
   }

   const {x, y} = state.prey;
   drawPixel('yellow', x, y);
```

```
        }

        window.onload = function () {
            state.canvas = document.querySelector('canvas');
            state.context = state.canvas.getContext('2d');

            window.onkeydown = function (e) {
                const direction = DIRECTIONS_MAP[e.key];

                if (direction) {
                    const [x, y] = direction;
                    if (-x !== state.direction.x
                            && -y !== state.direction.y)
                    {
                        state.direction.x = x;
                        state.direction.y = y;
                    }
                }
            }

            tick();
        };

    </script>
  </body>
</html>
```

3 in a Row code:

Código:

```
<!DOCTYPE html>
<!--
Juego del gato o 3 en raya creado en Html
-->
<html>
  <head>
    <meta charset="utf-8">
    <title>3 in a Row</title>
    <style type="text/css">
      body{
        width: 960px;
        margin: 0 auto;
      }
      h1{
        text-align: center;
        color: green;
      }
      #pantalla{
        border: groove 10px brown;
        background: lightblue;
      }
      #boton{
        background-color: red;
        color: white;
        font-size: 20px;
        text-align: center;
        font-weight: bolder;
        padding: 3px;
        border: solid 2px black;
      }
      #boton:hover{
        background-color: lightcoral;
        font-size: 22px;
      }

    </style>
    <script>
      var ctx, canvas;
      var fichas_array = new Array();
      var COLUMNAS = 3;
```

```
var RENGLONES = 3;
var fichas_X = 0;
var fichas_O = 0;
var tiradas = 0;
var gameOver = false;
var lados = 120;

window.onload = function () {
   /*Verificar canvas y ejecutarlo si esta bien o msj si error*/
   canvas = document.getElementById("pantalla");
   if (canvas && canvas.getContext) {
      ctx = canvas.getContext("2d");
      if (ctx) {
         /*Si canvas se ejecuto bien*/
         gato();
         mensaje("Pulse su jugada.");
         canvas.addEventListener("click", seleccionUsuario, false);
      } else {
         /*Si error*/
         alert("Error al crear el contexto!");
      }
   }
}

function gato() {
   /*Estas funciones hacen que se cargue la imagen del tablero en el canvas*/
   var imagen = new Image();
   function procesaImagen() {
      ctx.drawImage(imagen, 0, 0);
   }
   imagen.src = "tablero.png";
   imagen.onload = function (e) {
      procesaImagen();
   }
   /*Insertando fichas en array y las dibujamos   F,R,C */
   fichas_array.push(new Ficha(288, 29, lados, lados, 0, 0, 0));
   fichas_array.push(new Ficha(423, 29, lados, lados, 1, 0, 1));
   fichas_array.push(new Ficha(556, 29, lados, lados, 2, 0, 2));
   fichas_array.push(new Ficha(288, 160, lados, lados, 3, 1, 0));
   fichas_array.push(new Ficha(420, 160, lados, lados, 4, 1, 1));
   fichas_array.push(new Ficha(560, 160, lados, lados, 5, 1, 2));
   fichas_array.push(new Ficha(286, 295, lados, lados, 6, 2, 0));
   fichas_array.push(new Ficha(424, 295, lados, lados, 7, 2, 1));
```

```
    fichas_array.push(new Ficha(560, 295, lados, lados, 8, 2, 2));
}

function mensaje(cadena) {
    var lon = (canvas.width - (20 * cadena.length)) / 2;
    ctx.strokeStyle = "blue";
    ctx.clearRect(0, 420, canvas.width, 100);
    ctx.font = "bold 40px Courier";
    ctx.fillText(cadena, lon, 470);
}


function Ficha(x, y, w, h, i, ren, col) {
    this.x = x;
    this.y = y;
    this.w = w;
    this.h = h;
    this.i = i;
    this.ren = ren;
    this.col = col;
    this.peso = 0;
    this.valor = "";
    this.pinta = pintaFicha;
}


function pintaFicha(valor) {
    this.valor = valor;
    ctx.font = "bold 100px Arial";
    ctx.fillStyle = "blue";
    ctx.fillText(valor, this.x + 30, this.y + 100, this.w, this.h);

}


function ajustar(xx, yy) {
    var posCanvas = canvas.getBoundingClientRect();
    var x = xx - posCanvas.left;
    var y = yy - posCanvas.top;
    return {x: x, y: y}
}
```

```
function seleccionUsuario(e) {
   /*Ajusta la posicion en coordenadas del click*/
   var pos = ajustar(e.clientX, e.clientY);
   var x = pos.x;
   var y = pos.y;
   /*Ciclo verificacion si se ha dado en el lugar correcto y si esta vacio*/
   var ficha;
   for (i = 0; i < fichas_array.length; i++) {
      ficha = fichas_array[i];
      /*Aqui se verifica si el click esta dentro del espacio correcto*/
      if (ficha.x > 0) {
         if ((x > ficha.x) &&
               (x < ficha.x + ficha.w) &&
               (y > ficha.y) &&
               (y < ficha.y + ficha.h)) {
            tiradas++;  //aumenta uno a las tiradas
            break;  //Saca del ciclo
         }
      }
   }
   /*comprobar si aun se puede pintar ficha*/
   if (i < fichas_array.length) {
      /*Pintando la jugada del usuario (si esta vacio el lugar)*/
      if (ficha.valor == "") {
         ficha.pinta("X");
         /*Lanza un timer para que tire la pc en un segundo*/
         setTimeout(tiraMaquina, 1000);
      }
   }
   /****Verificamos si ganamos*******/
   verificaRenglones(true);
   verificaColumnas(true);
   verificaDiagonal1(true);
   verificaDiagonal2(true);
   if (gameOver == false && tiradas < 9) {
      /***Si no ganamos****/
      mensaje("Pensando...");
      canvas.removeEventListener("click", seleccionUsuario, false);
   } else {
      /*****Si ganamos desactivamos el listener******/
      if (gameOver == false) {
         mensaje("Empatados!!!");
      }
```

```
    }
}


function verificaFin(O, X) {
    fin = false;
    if (X == 3) {
        fin = true;
        mensaje("Felicidades, has ganado!!!");
        canvas.removeEventListener("click", seleccionUsuario, false);
    } else if (O == 3) {
        fin = true;
        mensaje("Lo siento, ha Perdido.");
        canvas.removeEventListener("click", seleccionUsuario, false);
    }
    return fin;
}


function buscaFicha(i, j) {
    for (k = 0; k < fichas_array.length; k++) {
        ficha = fichas_array[k];
        if (ficha.ren == i && ficha.col == j) {
            break;
        }
    }
    return ficha;
}


function verificaRenglones(calculaPeso) {
    if (gameOver == false) {
        for (i = 0; i < RENGLONES; i++) {
            fichas_X = 0;
            fichas_O = 0;
            for (j = 0; j < COLUMNAS; j++) {
                ficha = buscaFicha(i, j);
                fichas_X += (ficha.valor == "X" ? 1 : 0);
                fichas_O += (ficha.valor == "O" ? 1 : 0);
            }
            if (calculaPeso) {
                for (j = 0; j < COLUMNAS; j++) {
                    ficha = buscaFicha(i, j);
```

```
                pesoFicha(ficha.i, fichas_O, fichas_X);
            }
        }
        gameOver = verificaFin(fichas_O, fichas_X);
        if (gameOver)
            break;
    }
  }
}
/********************/

/****************Verificar Columnas*******************/
function verificaColumnas(calculaPeso) {
  if (gameOver == false) {
    for (j = 0; j < COLUMNAS; j++) {
        fichas_X = 0;
        fichas_O = 0;
        for (i = 0; i < RENGLONES; i++) {
            ficha = buscaFicha(i, j);
            fichas_X += (ficha.valor == "X" ? 1 : 0);
            fichas_O += (ficha.valor == "O" ? 1 : 0);
        }
        if (calculaPeso) {
            for (i = 0; i < RENGLONES; i++) {
                ficha = buscaFicha(i, j);
                pesoFicha(ficha.i, fichas_O, fichas_X);
            }
        }
        gameOver = verificaFin(fichas_O, fichas_X);
        if (gameOver)
            break;
    }
  }
}
/*****************************************************/

/*******Verificar diagonal desendiente********////
function verificaDiagonal1(calculaPeso) {
  if (gameOver == false) {
    fichas_X = 0;
    fichas_O = 0;
    for (i = 0; i < RENGLONES; i++) {
        ficha = buscaFicha(i, i);
```

```
            fichas_X += (ficha.valor == "X" ? 1 : 0);
            fichas_O += (ficha.valor == "O" ? 1 : 0);
        }
        if (calculaPeso) {
            for (i = 0; i < RENGLONES; i++) {
                ficha = buscaFicha(i, i);
                pesoFicha(ficha.i, fichas_O, fichas_X);
            }
        }
        gameOver = verificaFin(fichas_O, fichas_X);
    }
}
/**************************************************/

/*********Verificar Diagonal Acendente*****************/
function verificaDiagonal2(calculaPeso) {
    if (gameOver == false) {
        fichas_X = 0;
        fichas_O = 0;
        j = 2;
        for (i = 0; i < RENGLONES; i++) {
            ficha = buscaFicha(i, j);
            fichas_X += (ficha.valor == "X" ? 1 : 0);
            fichas_O += (ficha.valor == "O" ? 1 : 0);
            j--;
        }
        if (calculaPeso) {
            j = 2
            for (i = 0; i < RENGLONES; i++) {
                ficha = buscaFicha(i, j);
                pesoFicha(ficha.i, fichas_O, fichas_X);
                j--;
            }
        }
        gameOver = verificaFin(fichas_O, fichas_X);
    }
}

function pesoFicha(i, fichas_O, fichas_X) {
    ficha = fichas_array[i];
    if (ficha.valor == "") {
        if (fichas_O == 2 && fichas_X == 0) {
            ficha.peso += 10;
```

```
        } else if (fichas_O == 0 && fichas_X == 2) {
            ficha.peso += 6;
        } else if (fichas_O == 1 && fichas_X == 0) {
            ficha.peso += 3;
        } else {
            ficha.peso += 1;
        }
    } else {
        ficha.peso = 0;
    }
}

function tiraMaquina() {
    tiradas++;
    console.log("Tirada numero: " + tiradas);
    /*Comprobar si hay ganador*/
    if (gameOver == false) {
        /*Verificar con el peso*/
        verificaRenglones(true);
        verificaColumnas(true);
        verificaDiagonal1(true);
        verificaDiagonal2(true);
        /*Seleccionar mejor jugada*/
        fichas_X = 0;
        fichas_O = 0;
        mejorJugada = 0;
        for (i = 0; i < fichas_array.length; i++) {
            ficha = fichas_array[i];
            if (ficha.peso > mejorJugada) {
                mejorJugada = ficha.peso;
                ii = i;
            }
        }
        /*Realizar jugada de pc*/
        ficha = fichas_array[ii];
        ficha.pinta("O");
        verificaRenglones(false);
        verificaColumnas(false);
        verificaDiagonal1(false);
        verificaDiagonal2(false);
        /*Verificamos si alguien gano*/
        if (gameOver == false) {
            /*Si nadie gano activa el listener de click*/
```

```
                if (tiradas < 9) {
                    canvas.addEventListener("click", seleccionUsuario, false);
                    mensaje("Pulse su jugada...");
                }
            }
        }
    }


    </script>
  </head>
  <body>
    <h1>"3 in a Row"
      <p>"(Juego del gato)"</h1>
    <canvas id="pantalla" width="960px" height="500px">
      Tu navegador no soporta Canvas.
    </canvas>
    <button id="boton" type="reset" onclick="javascript:window.location.reload();">Volver a
Jugar</button>
  </body>
</html>
```

Glossary of Concepts

- Aplicación Web.- Es un conjunto de páginas que funcionan en internet; éstas páginas son las que el usuario ve a través de un navegador de internet (Internet Explorer de Microsoft,Chrome, Mozilla Firefox, etc.).

- Computadora Cliente.- Computadora que se conecta al servidor vía internet.

- Computadora Servidor.- Es es una computadora que se encarga de que las aplicaciones Web sean accesibles a través de internet.

- Controlador.- Es el software que procesa las peticiones del usuario. Decide qué modulo tendrá el control para que ejecute la siguiente tarea.

- Framework.- Se traduce al español como marco de trabajo y es un esqueleto para el desarrollo de una aplicación. Los frameworks definen la estructura de la aplicación, es decir, la manera en la que se organizan los archivos e inclusive, los nombres de algunos de los archivos y las convenciones de programación.
- HTTP.- HiperText Transfer Protocol, Protocolo de Transferencia de HiperTexto. Es un protocolo mediante el cual el software servidor se comunica con el navegador instalado en la computadora cliente.

- JSP.- JavaServer Page es una página HTML a la que se le incrusta código Java.

- Modelo.- Contiene el núcleo de la funcionalidad, es decir, ejecuta la “lógica del negocio”. Se le llama lógica del negocio a la forma en la que se procesa la información para generar los resultados esperados. El modelo se conecta a la base de datos para guardar y recuperar información.

- Página Web dinámica.- Es una página Web en la cual el servidor procesa la información proporcionada por el usuario y muestra los resultados de este procesamiento.

- Página Web estática.- Es una página Web informativa, en la que el usuario no puede hacer modificaciones.

- Servlet.- Es una clase Java (hija de la clase HttpServlet) que corre en el servidor.

- Software Servidor.- Es el software que controla la ejecución de aplicaciones Web, y corre en la Computadora Servidor.

- Vista.- Son los módulos SW involucrados en la interfaz con el usuario.

Suggestions

- Usar preferentemente los navegadores de Edge o Chrome para una mejor adaptación de la página.
- El zoom se recomienda tenerlo a un 90% -100%.
- Mantener abierto el servidor.
- Para regresar al menú principal, es necesario usar las flechas del navegador ubicadas en el superior izquierdo, usado para regresar.
- Mantener las clases en su lugar, descargar y ubicar correctamente.
- En caso de errores comuníquese con el propietario, datos en la portada.
- Usar librerías por defecto y versiones específicas requeridas.
- Apache Tomcat es una opción, también se puede usar con Glassfish 4.1.1
- Abrir Xampp para un mejor manejo de los servidores.

Tags html:

Tabla II-1. Tags básicos de HTML

Tag	Descripción
<!doctype>	Identifica el tipo de documento HTML
<html> </html>	Marca el inicio y el final del documento HTML.
<head> </head>	Marca el inicio y final del encabezado del documento HTML.
<title> </title>	El texto que se incluya es el que aparecerá en la barra de título del navegador.
<head> </head>	Marca el inicio y final del encabezado del documento HTML.
<body> </body>	Marca el inicio y el final del cuerpo del documento HTML.
<h1> </h1>	El texto tendrá el formato de "encabezado 1"
<h2 > </h2>	El texto tendrá el formato de "encabezado 2"

References

Para creación de servlet: